

Publive MCP Server

From "Add Custom Connector" to a tool result — every URL hit, function called, database CRUD, session transition, and New Relic span along the path.

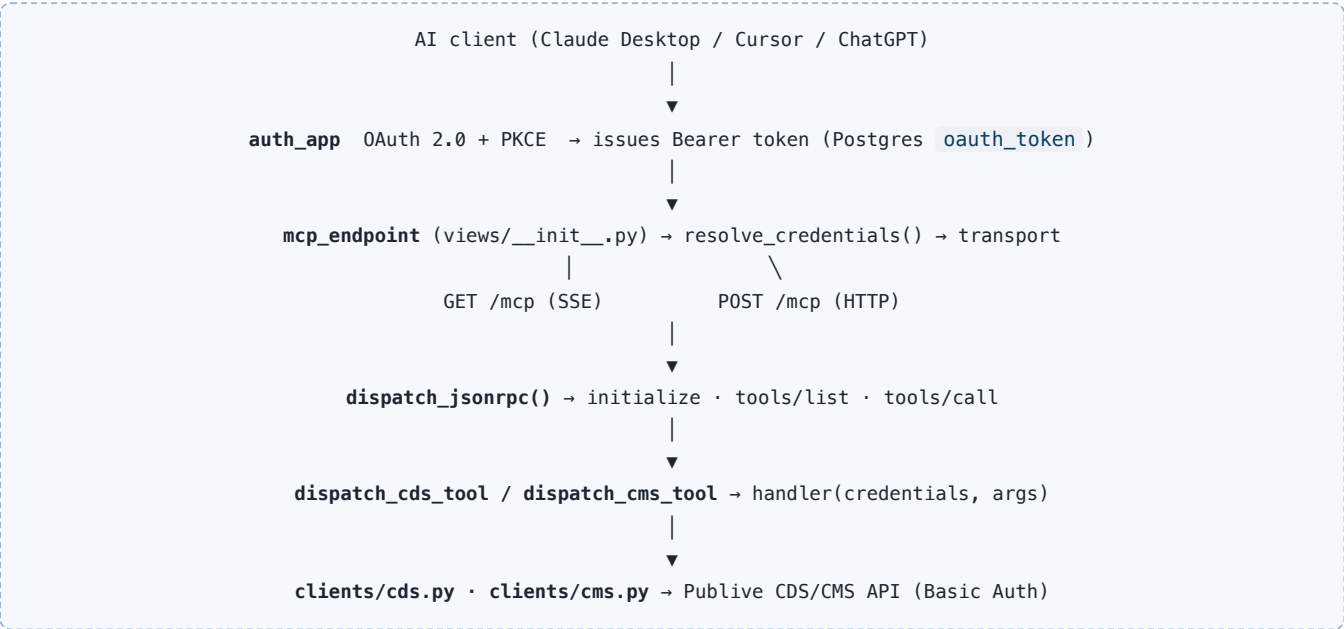
Contents

1. System overview & the two clocks of a request
2. Phase 1 — Discovery & OAuth 2.0 + PKCE connection
3. Phase 2 — Credential resolution on every request
4. Phase 3 — Transport: SSE vs. Streamable HTTP
5. Phase 4 — JSON-RPC dispatch (initialize / tools/list / tools/call)
6. Phase 5 — Tool execution & upstream API CRUD
7. Database CRUD reference
8. Session & state management (Redis)
9. New Relic observability — how everything is wrapped
10. Appendix — full URL map

Django + gunicorn (`-w 1 --threads 50`) · Redis-backed sessions · Railway
Protocol `2024-11-05` · 22 CDS read tools + 39 CMS write tools
Generated 2026-06-14

1 · System overview

The server is a thin, data-driven router. `views/__init__.py` authenticates every request once, then delegates to a transport module, which delegates to the JSON-RPC dispatcher, which routes by tool name to a handler, which calls the upstream Publive CDS/CMS HTTP API. No business logic lives in the view layer.



Layer	Module	Responsibility
Auth	<code>auth_app/</code>	OAuth PKCE + browser session login; validates credentials against CDS before issuing anything
Routing	<code>mcp_app/views/</code>	Authenticate once, dispatch to transport by HTTP method
Transport	<code>transport/sse.py</code> , <code>http.py</code>	SSE long-lived stream vs. stateless POST
Protocol	<code>protocol/dispatch.py</code>	JSON-RPC router + schema validation + per-call observability
Tools	<code>cds/</code> (22 read), <code>cms/</code> (39 write)	Schema + handler per tool, aggregated & dispatched by name
Clients	<code>clients/cds.py</code> , <code>cms.py</code>	HTTP calls to Publive APIs with Basic Auth, timeouts, retry
State	Redis	SSE session registry, message queues, per-session stats, rate limits
Storage	Postgres / SQLite	<code>oauth_client</code> , <code>oauth_code</code> , <code>oauth_token</code> + Django sessions

2 · Phase 1 — "Add Custom Connector" → OAuth 2.0 + PKCE

When a user adds the connector URL (`https://<host>/mcp`) in Claude Desktop, the client first probes for protection metadata, then runs the full PKCE handshake. Each step below names the **URL**, the **view function**, and the **DB CRUD** performed.

1

Probe — who guards this resource?

Client hits `GET /.well-known/oauth-protected-resource` → `oauth_protected_resource()` . Returns `{resource, authorization_servers:[base]}` . Triggered because the first unauthenticated `GET /mcp` returns `401` with a `WWW-Authenticate` header pointing here (`build_unauthorized_response()`). DB: none.

2 Discover endpoints

GET /.well-known/oauth-authorization-server (and /openid-configuration) → `oauth_server_metadata()`.
Advertises `authorize`, `token`, `register`, `revoke`, `userinfo` endpoints; S256 PKCE only;
`token_endpoint_auth_methods: none`. DB: none.

3 Dynamic client registration CREATE

POST /register → `oauth_register()`. Checks `Origin` (`check_origin`) and that `redirect_uri` is HTTPS or loopback (`is_registrable_redirect_uri`, RFC 8252). Mints `client_id = secrets.token_urlsafe(24)`.
DB CREATE: `OAuthClient.objects.create(client_id, redirect_uri)` → row in `oauth_client`.

4 Authorization — user enters Publive credentials CREATE CDS verify

Browser opens GET /authorize → `oauth_authorize()` renders `authorize.html`. User submits Publisher ID / API Key / API Secret via POST /authorize. `_validate_authorize_request()` re-checks client + redirect (auto-registers an unknown `client_id` if the redirect is acceptable), then `validate_cds_credentials()` calls the real CDS GET /posts/?limit=1 with Basic Auth — only a 2xx proceeds.
DB READ: `OAuthClient.objects.get(client_id)`. **DB CREATE:** `OAuthCode.objects.create(...)` — single-use code, 10-min expiry, stores {`publisherId`, `apiKey`, `apiSecret`} + the PKCE `code_challenge`. Then 302 redirect back with `?code=...&state=...`.

5 Token exchange (PKCE verify) READ DELETE CREATE

POST /token (grant `authorization_code`) → `oauth_token()`. Looks up the code, rejects if expired, verifies PKCE: `base64url(SHA256(code_verifier)) == stored code_challenge`, checks `redirect_uri`.
DB READ: `OAuthCode.objects.get(code)`. **DB DELETE:** `auth_code.delete()` (single use). **DB READ/CREATE:** upserts `OAuthToken` — reuses the existing token for the same client+publisher (stable identity) or creates a new one with `token` + `refresh_token`.
Returns {`access_token`, `token_type`:"bearer", `refresh_token`}.

6 Refresh / revoke (lifecycle)

POST /token (grant `refresh_token`) atomically rotates the refresh token under `select_for_update()` and returns the same access token. POST /revoke (RFC 7009) deletes the matching token row and always returns 200. **DB UPDATE** (rotate) / **DB DELETE** (revoke).

Browser alternative (humans): /connect → POST /auth/login → `auth_login()` validates via CDS, then stores credentials in a Django `session cookie` (`set_session_credentials`) with `session_ttl_seconds = -1` (never self-expires; ends only at POST /auth/logout). No bearer token is involved on this path.

3 · Phase 2 — Credential resolution on every request

Both GET /mcp and POST /mcp enter the single front door `mcp_endpoint()`, which calls `resolve_credentials(request)` (`protocol/auth.py`) before any transport logic runs.

```
# protocol/auth.py - resolve_credentials()
auth_header = request.META["HTTP_AUTHORIZATION"]
if auth_header.startswith("Bearer "):
    # PKCE path → DB lookup
    token = OAuthToken.objects.get(token=value)          # DB READ
    creds = {**token.credentials, "publisherId": token.publisher_id}
    return creds, None, None
else:
    # Session path → cookie + server-side TTL
    creds = get_session_credentials(request.session)
    if check_session_ttl(request.session):                # expired?
        request.session.flush()                          # DB DELETE (session row)
        return None, None, "SESSION_EXPIRED"
    return creds, None, None
```

On failure → `build_unauthorized_response()` returns `401` with the RFC 6750 `WWW-Authenticate` header (kicking off step 1 of the OAuth flow). On success the resolved `credentials` dict — `{publisherId, apiKey, apiSecret}` — flows down through every subsequent layer and is ultimately what the upstream HTTP clients Basic-Auth with.

HTTP method	Routed to	Transport
GET /mcp	<code>sse_open()</code> → <code>open_sse_connection()</code>	Long-lived SSE stream
POST /mcp	<code>http_mcp()</code> → <code>handle_http_request()</code>	Stateless Streamable HTTP (must be <code>application/json</code> , else 415)
POST /mcp/message?sessionId=X	<code>sse_message()</code> → <code>handle_sse_message()</code>	Delivers a client message into an open SSE session

4 · Phase 3 — Transport layer

4.1 SSE transport (legacy 2024-11-05) — two URLs, possibly two processes

SSE splits a session across two HTTP requests. The stream is opened once with `GET /mcp`; every client message thereafter is a separate `POST /mcp/message`. Because the session registry and message queue live in **Redis** (not process memory), the GET stream and the POST can land on different gunicorn workers/replicas.

1

`GET /mcp` → `open_sse_connection()`
Generates `session_id = uuid4()`. `register_session()` does an atomic Redis `SET mcp:session:<id> + SADD mcp:active_sessions + SCARD`. If the active count exceeds `MCP_MAX_SSE_SESSIONS` (default 2) the gate rolls back (`close_session`) and returns `503` + `Retry-After` — a stream pins a thread for its whole life, so the cap protects POST throughput. Then it streams: first an `event: endpoint` line carrying the `POST /mcp/message?sessionId=...` URL, then loops `pop_message()` (Redis `BLPOP`, 25 s) emitting `event: message` frames or `: keepalive` pings.

2

`POST /mcp/message` → `handle_sse_message()`
Reads `sessionId`, looks up `get_session()` in Redis (→ `400 MCPSessionMissing` if absent), parses the JSON-RPC body, calls `dispatch_jsonrpc(...)`, then `push_message()` (Redis `RPUSH`, capped at `MCP_QUEUE_MAXSIZE` =100, 5 s wait) so the GET stream on whatever process holds it can deliver the response. Returns `{"ok": true}` to the POST.



4.2 Streamable HTTP transport — stateless, single URL

`POST /mcp` → `handle_http_request()` parses one JSON object (or a batch list), derives a stable `session_id` via `derive_session_id()` (Django session key → SHA-256 of bearer → transient UUID), calls `dispatch_jsonrpc()` per message and returns the response directly in the HTTP body. Notifications (no `id`) → `202`. No Redis session/queue needed — request and response share one connection.

5 · Phase 4 — JSON-RPC dispatch

Both transports converge on `dispatch_jsonrpc(body, credentials, request, session_id, token_expires_at)` in `protocol/dispatch.py`. It routes by the JSON-RPC `method`:

method	Handling
<code>initialize</code>	Returns <code>protocolVersion 2024-11-05</code> , <code>capabilities.tools</code> , <code>serverInfo</code> . Stores protocol version in the session.

<code>tools/list</code>	Returns <code>TOOLS</code> + <code>CMS_TOOLS</code> (all 61 schemas) — built once at import as a data-driven registry.
<code>tools/call</code>	The main path → <code>_handle_tool_call()</code> (below).
<code>ping</code>	Returns <code>{}</code> .
notifications (no <code>id</code>)	Returns <code>None</code> (no response emitted).
known-but-unimplemented (resources/*, prompts/*, sampling/*, ...)	JSON-RPC error <code>-32601</code> .
truly unknown	<code>-32601</code> + <code>MCPUnknownMethod</code> custom event.

5.1 Inside `_handle_tool_call()`

- 1 Prompt capture:** `extract_prompt_for_tool_call()` pulls the user's natural-language prompt from headers / `_meta` / args (the `_prompt` arg is stripped before the tool runs), and — if under the cluster-wide rate budget (`should_emit_prompt_event()` , Redis fixed-window) — emits an `MCPPrompt` event.
- 2 Schema validation:** `_validate_tool_args()` checks required fields, JSON types, and `minLength` against the tool's `inputSchema`. A failure short-circuits to an `isError` result — the tool never runs.
- 3 Session timeline & rate limits:** reads/updates per-session stats in Redis; CMS writes are capped at 100 ops/session per bucket (`create` vs `update_delete`).
- 4 Dispatch:** `dispatch_cms_tool()` if the name is in `CMS_TOOL_NAMES`, else `dispatch_cds_tool()`. Timed with `time.perf_counter()`.
- 5 Classify result:** success, *degraded* (handler returned an `error` / `error_type` dict), or *error* (exception). Each path records metrics + a custom event and updates session counters, then returns an MCP `content` list.

6 · Phase 5 — Tool execution & upstream API CRUD

Each tool module exports `SCHEMAS` + `HANDLERS`; the package `__init__.py` aggregates them into a name→handler registry. `dispatch_cds_tool()` / `dispatch_cms_tool()` resolve the handler, track concurrency, wrap it in a New Relic function trace, and call `handler(credentials, args)`.

```
# cds/__init__.py - dispatch_cds_tool()
handler = _HANDLER_REGISTRY.get(name)          # name → callable
with fn_trace(name, group="Tool"):
    return handler(credentials, args)           # e.g. fetch_published_posts
# HTTP 401 from upstream → returns a friendly "auth_expired" degraded dict
```

6.1 The HTTP clients (the actual CRUD against Publive)

Handlers never speak HTTP directly — they call `clients/cds.py` (read) or `clients/cms.py` (write). Both build the base URL as `https://{cds|cms}-beta.thepublive.com/publisher/{publisherId}` and authenticate with HTTP Basic (`apiKey:apiSecret`).

CRUD	Client fn	HTTP	Notes
READ	<code>cds_get()</code>	GET	5 s timeout, 1 retry on timeout / HTTP 408
READ	<code>cms_get()</code>	GET	10 s timeout, no retry
CREATE	<code>cms_post()</code>	POST	Tier 2: <code>dry_run=true</code> by default → preview, no write
UPDATE	<code>cms_patch()</code>	PATCH	Tier 3: <code>dry_run=true</code> shows old→new diff

DELETE	<code>cms_delete()</code>	DELETE	Tier 3: requires <code>dry_run=false</code> AND <code>confirm_delete=true</code>
--------	---------------------------	--------	--

CMS client functions return either parsed JSON or a normalized error dict (`{error_type, message, retryable}`) — the dispatcher treats the latter as a "degraded" result rather than an exception.

Write safety (tiered model): the dry-run/confirm guard lives in the handlers via `cms/helpers.py` (`preview_create_op` , `preview_update_op` , `DELETION_REQUIRES_CONFIRMATION`). This is what makes destructive operations a two-call ceremony rather than a single mistake.

7 · Database CRUD reference

The Django ORM owns three OAuth tables plus the Django session table. Live CDS/CMS content is *not* in this database — it lives behind the Publive API. Credentials are stored as **plain JSON** (encryption was deliberately removed).

Table / model	Created	Read	Updated	Deleted
<code>oauth_client</code> (<code>OAuthClient</code>)	<code>/register</code> ; auto-reg in <code>/authorize</code>	<code>/authorize</code> validation	—	—
<code>oauth_code</code> (<code>OAuthCode</code>)	POST <code>/authorize</code> after CDS verify	POST <code>/token</code>	—	On exchange & on expiry (single-use, 10 min)
<code>oauth_token</code> (<code>OAuthToken</code>)	POST <code>/token</code> (new)	Every Bearer request (<code>resolve_credentials</code>)	Refresh-token rotation; <code>refresh_token</code> backfill	<code>/revoke</code>
Django session	<code>/auth/login</code>	Every cookie request	(rolling save disabled)	<code>/auth/logout</code> , TTL expiry flush

8 · Session & state management (Redis)

"Session" means two different things here, and that distinction is the crux of the architecture.

Kind	Where	Lifetime	Purpose
Auth session (browser)	Django session cookie + session table	<code>ttl=-1</code> ; ends at <code>/auth/logout</code>	Holds Publive credentials for cookie clients
MCP SSE session	Redis (<code>mcp:session:<id></code> + <code>mcp:active_sessions</code> set)	24 h TTL, refreshed on access; closed on disconnect	Lets GET stream & POST message land on different processes
Message queue	Redis list <code>mcp:session_queue:<id></code>	24 h TTL; deleted on close	RPUSh (producer) / BLPOP (consumer) hand-off
Session stats	Redis hash (<code>redis_session_stats</code>)	Popped at close	Tool counts, durations, token estimates, tool sequence
Rate-limit counters	Redis (<code>INCR</code> /fixed-window)	Per session / per minute	CMS write caps; MCPPrompt event budget

Why Redis (the "-w 1" story): the SSE registry, queues, stats and rate limits were originally in-process dicts, which pinned the app to exactly one gunicorn worker. Externalizing all four to Redis removes that constraint — though `-w 1 --threads N` stays pinned intentionally during the staged rollout. On SSE close, `_close_sse_session()` tears down the session + queue and emits `MCPSessionSummary` / `SSESessionClose` (and `MCPSessionAbandoned` if zero tool calls).

9 · New Relic observability — how everything is wrapped

Every New Relic call goes through guarded wrappers in `nr_utils.py`. If the agent isn't installed (`import newrelic.agent` fails) or there's no active transaction, each helper becomes a no-op — so the app runs cleanly with NR absent.

```
# nr_utils.py – the guard pattern
try:
    import newrelic.agent as _nr
except ImportError:
    _nr = None

def add_attrs(pairs):
    if _current_transaction():
        # no-op when NR absent
        _nr.add_custom_attributes(pairs)
```

9.1 Three wrapping mechanisms

- **Function traces (decorators):** `@newrelic.agent.function_trace(...)` on `mcp_endpoint`, `handle_http_request`, `handle_sse_message`, `dispatch_cds_tool`, `cds_get`, and every auth view — these build the trace waterfall.
- **Transaction naming + attributes:** `set_txn_name()` groups transactions (`MCP/initialize`, `MCP/<tool>`, `CDS/<path>`, `Auth/*`); `add_attrs()` attaches `mcp.*` / `cds.*` / `auth.*` custom attributes.
- **Custom events & metrics:** `record_event()` / `record_metric()` emit the events below; `get_linking_metadata()` correlates events to the APM trace via `trace.id` / `span.id`.

9.2 Key custom events

MCPPrompt
MCPToolError
MCPToolDegraded
MCPUnknownMethod
SSESessionOpen
SSESessionClose
MCPSessionAbandoned
MCPSessionMissing
MCPSessionSummary

SSE transactions additionally call `suppress_apdex()` + `suppress_trace()` so long-lived streams don't flood the slow-transaction list or skew Apdex. `notice_err()` reports exceptions with an `error.layer` tag (`transport` / `auth` / `cds` / `tool`) so failures are attributable to a layer.

10 · Appendix — full URL map

URL	View	Purpose
GET /	health_check	Liveness
GET /mcp	mcp_endpoint → sse_open	Open SSE stream
POST /mcp	mcp_endpoint → http_mcp	Streamable HTTP JSON-RPC
POST /mcp/message	sse_message	Deliver msg into SSE session
/.well-known/oauth-protected-resource	oauth_protected_resource	RFC 9728 metadata
/.well-known/oauth-authorization-server /.well-known/openid-configuration	oauth_server_metadata	AS metadata
POST /register	oauth_register	Dynamic client registration
GET/POST /authorize, /oauth/authorize	oauth_authorize	Auth form + code issuance
POST /token, /oauth/token	oauth_token	Code→token, refresh
POST /revoke, /oauth/revoke	oauth_revoke	RFC 7009 revocation
GET /userinfo	oauth_userinfo	OIDC identity (sub = publisherId)
GET /connect	connect	Browser login page
POST /auth/login	auth_login	Session login (CDS-verified)
GET /auth/status	auth_status	Session state + TTL
POST /auth/logout	auth_logout	Flush session